

## Exercice et corrigé

### Simulation & machine learning

Quatre parties pour simuler et apprendre des données : l'erreur du Monte Carlo, le bootstrap et la VaR, la préparation des données pour le ML, et l'évaluation d'un classifieur. Traitez chaque partie avant de lire la solution.

#### Partie A · L'erreur du Monte Carlo

##### ÉNONCÉ

Une simulation Monte Carlo avec  $b = 8\,000$  réplifications donne une erreur-type de 0,60. (a) Combien de réplifications pour ramener l'erreur-type à 0,15 ? (b) Pourquoi la précision progresse-t-elle si lentement ?

##### SOLUTION

- (a) On veut diviser l'erreur par 4 ( $0,60 \rightarrow 0,15$ ). Comme  $s.e. \propto 1/\sqrt{b}$ , il faut multiplier  $b$  par  $4^2 = 16$ , soit **128 000** réplifications.
- (b) Le CLT donne  $s.e. = \sigma/\sqrt{b}$  : l'erreur décroît en  $1/\sqrt{b}$ , pas en  $1/b$ . Diviser l'erreur par 2 exige donc 4× plus de réplifications, et un facteur 10 de précision en réclame 100×. C'est le coût de la racine carrée, à connaître pour dimensionner une simulation.

#### Partie B · Bootstrap et VaR

##### ÉNONCÉ

Un risk manager estime une VaR annuelle du S&P 500 par bootstrap. (a) Doit-il tirer avec ou sans remise, et pourquoi les répétitions ne sont-elles pas une erreur ? (b) Les rendements présentent du volatility clustering. Faut-il un iid bootstrap ou un CBB, et quelle taille de bloc ? (c) Quelle limite fondamentale empêche le bootstrap de prévoir une crise inédite ?

##### SOLUTION

- (a) **Avec remise** : la même observation peut revenir plusieurs fois, ce qui est normal (la probabilité d'au moins une répétition est quasi certaine dès qu'on multiplie les échantillons). C'est ce tirage avec remise qui permet de fabriquer un grand nombre d'échantillons à partir d'un seul jeu de données.
- (b) Un **CBB** (circular block bootstrap), car les données sont dépendantes : il tire des blocs de  $q$  observations consécutives et conserve les grappes de volatilité. Taille de bloc  $q \approx \sqrt{n}$ . Un iid bootstrap casserait les grappes et sous-estimerait la VaR extrême.
- (c) Le bootstrap rééchantillonne l'historique : il ne peut **jamais générer une perte pire que la plus grande déjà observée** (Black Swan). Il mélange aussi tous les régimes, ce qui le rend trompeur quand le marché entre dans un état anormal. Un bon modèle de simulation peut, lui, extrapoler au-delà de l'historique.

#### Partie C · Préparer les données pour le ML

##### ÉNONCÉ

(a) Une feature a une moyenne de 40 et un écart-type de 5. Standardisez l'observation  $x = 52$ . (b) Quand préfère-t-on la standardization à la normalization min-max ? (c) Rééchelonne-t-on aussi le target ?

##### SOLUTION

- (a)  $\tilde{x} = (x - \mu)/\sigma = (52 - 40)/5 = 12/5 = \mathbf{2,4}$  : l'observation est à 2,4 écarts-types au-dessus de la moyenne. Après standardization, la feature a une moyenne de 0 et une variance de 1.
- (b) On préfère la standardization **en présence d'outliers** : la normalization min-max (division par  $\max - \min$ ) tasserait alors le gros des observations dans une plage étroite, peu représentative de leur dispersion réelle.
- (c) **Non** : on rééchelonne uniquement les **features** (inputs), jamais le **target** (la variable à prédire). Le rescaling sert à mettre les features à la même échelle pour que les distances et les pénalités les comparent équitablement.

## Partie D · Évaluer un classifieur

### ÉNONCÉ

Un modèle de détection de fraude donne, sur 500 transactions : TP = 180, FN = 60, FP = 40, TN = 220. (a) Calculez accuracy, precision et recall. (b) Une AUC de 0,5 est-elle un score « moyen » ? (c) Le coût d'une fraude non détectée est très élevé : comment ajuster le seuil Z, et au prix de quoi ?

### SOLUTION

1. (a) Accuracy =  $(180+220)/500 = \mathbf{80,0\%}$ . Precision =  $TP/(TP+FP) = 180/220 = \mathbf{81,8\%}$ . Recall =  $TP/(TP+FN) = 180/240 = \mathbf{75,0\%}$ .
2. (b) **Non** : une AUC de 0,5 correspond à la diagonale du hasard, soit **aucun pouvoir prédictif**. AUC = 1 est un classifieur parfait ; une AUC inférieure à 0,5 fait pire que le hasard.
3. (c) Un faux négatif (fraude non détectée) coûte cher : on **abaisse le seuil Z** (par exemple à 0,2 au lieu de 0,5) pour prédire « fraude » dès qu'il existe un risque même faible. Cela remonte le recall (moins de fraudes ratées), au prix d'une **precision plus faible** (plus de fausses alertes) : c'est l'arbitrage que trace la courbe ROC.

À retenir : l'erreur Monte Carlo décroît en  $1/\sqrt{b}$  ( $\div 2 \Rightarrow \times 4$  réplications) ; le bootstrap tire avec remise et ne crée jamais de perte inédite (CBB si dépendance, blocs  $\sqrt{n}$ ) ; on rescale les features (pas le target), standardization si outliers ; et un classifieur se juge par precision, recall et AUC (0,5 = aucun pouvoir prédictif), le seuil Z arbitrant entre faux positifs et faux négatifs.